

Healthcare Management System REST API

Project Documentation

Project Name: Healthcare Management System REST API

Technology Stack:

- Java 21
 - Spring Boot 4.1.1 / 3.x
 - Spring Data JPA (Hibernate)
 - MySQL / H2 Relational Database
 - Maven (Dependency Management)
 - Lombok
 - Jakarta Validation
 - Spring Security (Basic Auth)
 - Springdoc OpenAPI (Swagger UI)
 - Postman (API Testing & Verification)
-

1. Project Overview

The **Healthcare Management System REST API** is an enterprise-grade backend service built using **Spring Boot**. It enables complete digital administration of modern healthcare operations across six vital modules: **Patients, Doctors, Appointments, Prescriptions, Medical Records, and Billing**.

The system adheres strictly to a clean, decoupled **Layered Architecture** comprising Controller, Service, Repository, and Entity layers. This ensures high testability, separation of business concerns, robust input validation, centralized exception handling, and enterprise database integration.

2. Objectives

- **Patient Administration:** Maintain demographic records, contact info, and registration history.
- **Doctor Management:** Store physician profiles, specialties, and professional contact details.
- **Appointment Scheduling:** Connect patients and doctors for scheduled medical consultations.
- **Prescription Issuance:** Digitally track medications, dosages, and administration instructions.
- **Medical Record Tracking:** Maintain comprehensive diagnostic histories and treatment plans.
- **Billing & Finance:** Record invoices, billing amounts, and payment statuses per consultation.
- **Enterprise REST APIs:** Expose full CRUD endpoints adhering to standard HTTP status codes.
- **Relational Data Integrity:** Model 1:Many and 1:1 foreign key relationships using Spring Data JPA.

3. Technologies Used

Technology	Purpose
Java 21	Primary object-oriented programming language
Spring Boot	Rapid enterprise backend application framework
Spring Data JPA	Data persistence, Hibernate ORM, repository abstraction
MySQL / H2	Relational database engines for storage and testing
Maven	Build automation and dependency management tool
Lombok	Eliminates boilerplate code (@Data, @Builder, etc.)
Jakarta Validation	Declarative request body validation (@NotBlank, @Email)
Spring Security	API authentication and endpoint authorization
Springdoc OpenAPI	Interactive live Swagger UI documentation
Postman	Automated REST API testing and validation suite

4. Project Architecture

The system implements a production-grade 4-layer architectural design pattern:

[illegible]

5. Project Structure

HealthcareManagementSystem ■■■■ controller ■■■■ PatientController.java ■■■■
DoctorController.java ■■■■ AppointmentController.java ■■■■ PrescriptionController.java ■■■■
MedicalRecordController.java ■■■■ BillingController.java ■■■■ entity ■■■■
Patient.java ■■■■ Doctor.java ■■■■ Appointment.java ■■■■ Prescription.java ■■■■
MedicalRecord.java ■■■■ Billing.java ■■■■ repository ■■■■ PatientRepository.java ■■■■
DoctorRepository.java ■■■■ AppointmentRepository.java ■■■■ PrescriptionRepository.java ■■■■
MedicalRecordRepository.java ■■■■ BillingRepository.java ■■■■ service ■■■■
PatientService.java ■■■■ DoctorService.java ■■■■ AppointmentService.java ■■■■
PrescriptionService.java ■■■■ MedicalRecordService.java ■■■■ BillingService.java ■■■■
exception ■■■■ ResourceNotFoundException.java ■■■■ GlobalExceptionHandler.java ■■■■ config
■■■■ SecurityConfig.java ■■■■ HealthcareApplication.java

6. Database Design

The database contains six interconnected entities designed with normalized schemas:

Patient Entity

Field	Type	Constraint / Details
id	Long	Primary Key, Auto-increment
name	String	@NotBlank
dob	LocalDate	Date of birth
gender	String	Male / Female / Other
contact	String	@NotBlank, Phone number
address	String	Residential address
createdAt	LocalDateTime	@CreationTimestamp

Doctor Entity

Field	Type	Constraint / Details
id	Long	Primary Key, Auto-increment
name	String	@NotBlank
specialization	String	@NotBlank (e.g. Cardiology)
phone	String	@NotBlank
email	String	@NotBlank, @Email

Appointment Entity

Field	Type	Constraint / Details
id	Long	Primary Key, Auto-increment

patient	Patient	Foreign Key (@ManyToOne)
doctor	Doctor	Foreign Key (@ManyToOne)
appointmentDate	LocalDateTime	@FutureOrPresent
reason	String	@NotBlank
status	String	SCHEDULED / COMPLETED

Prescription Entity

Field	Type	Constraint / Details
id	Long	Primary Key, Auto-increment
appointment	Appointment	Foreign Key (@OneToOne, Unique)
medication	String	@NotBlank, Drug name
dosage	String	@NotBlank (e.g. 10mg)
instructions	String	Dosage schedule / advice

Medical Record Entity

Field	Type	Constraint / Details
id	Long	Primary Key, Auto-increment
patient	Patient	Foreign Key (@ManyToOne)
diagnosis	String	@NotBlank, Clinical condition
treatment	String	@NotBlank, Medical action taken
recordDate	LocalDateTime	@CreationTimestamp

Billing Entity

Field	Type	Constraint / Details
id	Long	Primary Key, Auto-increment
appointment	Appointment	Foreign Key (@OneToOne, Unique)
amount	BigDecimal	@NotNull, @PositiveOrZero
status	String	PAID / UNPAID / PENDING
billingDate	LocalDateTime	@CreationTimestamp

7. Entity Relationships

Patient ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ 1 ■ 1 ■ 1 ▼ M ▼ M ■ Appointment
MedicalRecord ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ 1 1 ■ ▼ 1 ▼ M

Prescription Doctor ■ ▼ 1:1 Billing

Relationship Details:

- **Patient 1 → Many Appointments:** A patient can book multiple hospital visits over time.
- **Doctor 1 → Many Appointments:** A doctor conducts consultations with many patients.
- **Appointment 1 → 1 Prescription:** Each clinical consultation generates one specific prescription.
- **Patient 1 → Many Medical Records:** A patient accumulates multiple medical diagnosis entries.
- **Appointment 1 → 1 Billing:** Each consultation has exactly one associated billing invoice.

8. API Endpoints

Patient APIs

Method	Endpoint	Description
POST	/api/patients	Register new patient record
GET	/api/patients	Retrieve all registered patients
GET	/api/patients/{id}	Retrieve single patient by ID
PUT	/api/patients/{id}	Update patient details
DELETE	/api/patients/{id}	Remove patient from system

Doctor APIs

Method	Endpoint	Description
POST	/api/doctors	Register new doctor profile
GET	/api/doctors	List all active doctors
GET	/api/doctors/{id}	Get doctor profile by ID
PUT	/api/doctors/{id}	Update doctor information
DELETE	/api/doctors/{id}	De-register doctor profile

Appointment APIs

Method	Endpoint	Description
POST	/api/appointments	Schedule a consultation
GET	/api/appointments	List all scheduled appointments
GET	/api/appointments/{id}	Get appointment details by ID
PUT	/api/appointments/{id}	Reschedule / modify appointment

DELETE	/api/appointments/{id}	Cancel scheduled appointment
---------------	------------------------	------------------------------

Prescription APIs

Method	Endpoint	Description
POST	/api/prescriptions	Issue new prescription
GET	/api/prescriptions	List all prescriptions
GET	/api/prescriptions/{id}	Get prescription by ID
PUT	/api/prescriptions/{id}	Update medication or dosage
DELETE	/api/prescriptions/{id}	Delete prescription entry

Medical Record APIs

Method	Endpoint	Description
POST	/api/records	Record clinical diagnosis
GET	/api/records	List all clinical records
GET	/api/records/{id}	Retrieve record by ID
PUT	/api/records/{id}	Update diagnosis/treatment
DELETE	/api/records/{id}	Remove medical record

Billing APIs

Method	Endpoint	Description
POST	/api/billing	Generate invoice for appointment
GET	/api/billing	List all billing entries
GET	/api/billing/{id}	Get billing details by ID
PUT	/api/billing/{id}	Update amount or payment status
DELETE	/api/billing/{id}	Remove billing record

9. Validation Used

To ensure robust data sanitization and prevent corrupt records from entering the database, the application uses standard **Jakarta Bean Validation** annotations:

- **@NotBlank**: Enforces that strings (names, contacts, specializations, reasons) cannot be null or empty.
- **@NotNull**: Ensures foreign key relationship entities and critical numeric fields are present.
- **@Email**: Validates that doctor email addresses follow valid RFC-compliant syntax.
- **@FutureOrPresent**: Ensures scheduled appointment dates cannot be placed in the past.
- **@PositiveOrZero**: Guarantees that billing amounts cannot be negative values.
- **@Valid**: Triggers automatic validation cascades across controller request payloads.

10. Exception Handling

The project handles exceptions centrally across the application using a dedicated **@ControllerAdvice** class (**GlobalExceptionHandler**), ensuring standardized JSON error responses.

Custom Exception:

- **ResourceNotFoundException**: Thrown when an invalid ID is requested.

Global Exception Handler Handles:

- **Resource Not Found (404)**: Returned when queries for non-existent entities fail.
- **Validation Errors (400)**: Catches **MethodArgumentNotValidException** and returns detailed field-by-field error maps.
- **Internal Server Errors (500)**: Catches unhandled runtime exceptions with user-friendly error messages.

11. HTTP Status Codes

Status Code	Meaning	Scenario in Project
200 OK	Success	Successful GET, PUT, and general operations
201 Created	Created	Returned when a new entity is saved via POST
204 No Content	Deleted	Successful DELETE operation
400 Bad Request	Validation Failure	Missing required fields or invalid format
401 Unauthorized	Auth Required	Missing or invalid Basic Auth credentials
404 Not Found	Resource Missing	Entity with requested ID does not exist
500 Internal Error	Server Fault	Unhandled system or database exceptions

12. Features

- **Patient Management:** Complete CRUD lifecycle tracking for hospital patients.
- **Doctor Management:** Physician directory with specialty and contact categorization.
- **Appointment Scheduling:** Relational scheduling connecting patients with available doctors.
- **Prescription Generation:** Medication dosage and patient administration instructions.
- **Medical Record History:** Comprehensive diagnostic and clinical treatment logs.
- **Billing & Invoicing:** Payment status and financial records linked directly to appointments.
- **Relational Modeling:** Configured with JPA @ManyToOne and @OneToOne mappings.
- **Live Swagger Documentation:** Interactive API documentation playground at /swagger-ui.html.
- **Spring Security:** Protected endpoints using Basic Authentication.
- **Automated Database DDL:** Automatic table schema generation and updates via Hibernate.

13. Future Enhancements

- **JWT Token Authentication:** Stateless token-based security replacing HTTP Basic Auth.
- **Role-Based Access Control (RBAC):** Discrete roles for Patients, Doctors, and Administrators.
- **Telehealth Integration:** WebRTC video call links embedded within appointment records.
- **Automated SMS & Email Reminders:** Notifications for upcoming scheduled appointments.
- **Payment Gateway Integration:** Stripe or Razorpay integration for real-time invoice payments.
- **Cloud Storage (AWS S3):** Secure cloud hosting for lab reports and MRI/X-ray attachments.
- **Docker & CI/CD:** Containerized microservices deployment with automated GitHub Actions.

14. Conclusion

The **Healthcare Management System REST API** represents an enterprise-ready, robust backend solution designed for medical facilities and clinical administration. By applying modern Spring Boot principles, clean layered architecture, declarative validation, relational JPA mapping, and centralized exception handling, the project achieves high code maintainability, security, and scalability suitable for production deployment and technical interview demonstrations.